Each city belongs to a country. It seems reasonable to use option groups to group city options by country within the select element. When the user is creating a new property in the application, they can find the city where the property is located more easily. So let's add a new public property to the `City` class to represent the country name:

```
public string CountryName { get; set; }
```

Now you'll change the `GetCityOptions` method in the Pages\PropertyManager\Create .cshtml.cs file to assign a country name to a selection of cities and then pass in `nameof(City.CountryName)` to represent the `DataGroupField`.

Listing 6.16 Adding an option group to the city options

```
private SelectList GetCityOptions()
{
    var cities = new List<City>
    {
        new City{ Id = 1, Name = "Barcelona" , CountryName = "Spain" },
        new City{ Id = 2, Name = "Cadiz" , CountryName = "Spain" },
        new City{ Id = 3, Name = "London", CountryName = "United Kingdom" },
        new City{ Id = 4, Name = "Madrid" , CountryName = "Spain" },
        new City{ Id = 5, Name = "Rome", CountryName = "Italy" },
        new City{ Id = 6, Name = "Venice", CountryName = "Italy" },
        new City{ Id = 7, Name = "York" , CountryName = "United Kingdom" },
    };
    return new SelectList(cities, nameof(City.Id), nameof(City.Name),
    ➥ null, nameof(City.CountryName));
}
```

When you run the page, you should see the data grouped by country, with the country name set as the grouping label (figure 6.12). Most often, you will work in an object-oriented way with your data. You already have a `Country` class, so you can use composition to represent the relationship between cities and countries by adding a `Country` property to the `City` class (bolded in the following listing).